



Tecnológico de Monterrey

Estudiante:

Axel Javier Rosas Rodríguez | A01738607

Profesor:

Alfredo García Suárez

Nombre de la Institución:

Instituto Tecnológico y de Estudios Superiores de
Tecnológico de Monterrey

Materia:

Fundamentación de la Robótica

Fecha:

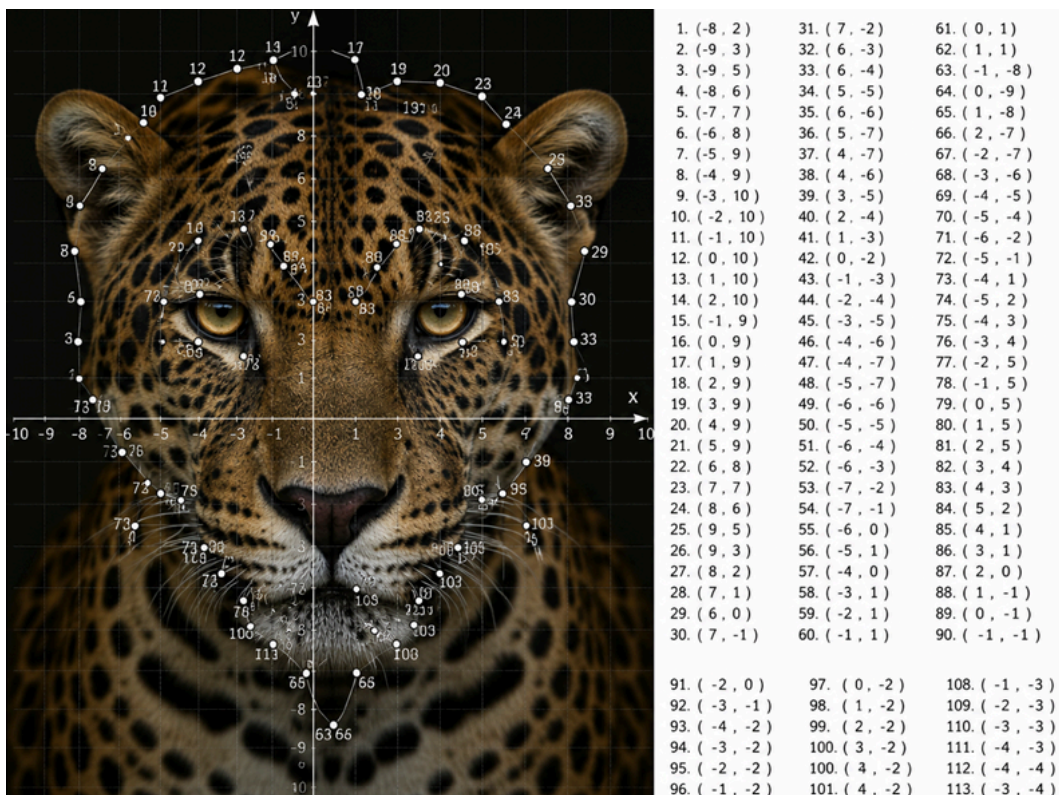
28 de Mayo de 2026

Título del trabajo:

Actividad 9 (Planeación de Trayectorias)

Descripción

1. Obtener el mapa de puntos para generar la planeación de trayectorias a partir del rostro del felino, marcando set-points con el software Geogebra.
2. Planear la trayectoria que debe seguir el algoritmo empleando el mapa de puntos obtenido, considerando la descripción más próxima a los rasgos físicos del rostro del felino (Forma del rostro, pelaje, ojos, nariz, mandíbula, orejas, etc.).
3. Implementar el código requerido para generar la planeación de trayectorias empleado la comparación de las 2 técnicas que consideres más eficiente para esta aplicación (Control en lazo abierto, control de posición, control de trayectorias ó PurePursuit (way-points)). Considerando el ajuste de los parámetros cinemáticos del robot móvil como: Velocidad angular (w), velocidad lineal (v), ancho de la muestra (t_s), tiempo de simulación (t), ganancias, etc.



Metodología

Fase 1: Mapeo Espacial y Extracción de Coordenadas

- **Importación y calco:** Cargar la imagen del rostro del felino en el software GeoGebra.
- **Generación de set-points:** Colocar puntos estratégicos sobre la imagen para delimitar con precisión la geometría de los rasgos físicos clave (contorno del rostro, pelaje, orejas, ojos, nariz y mandíbula).
- **Extracción de datos:** Exportar la lista de coordenadas espaciales (x, y) generadas para construir el mapa de puntos base.

Fase 2: Planeación de la Trayectoria

- **Secuenciación de la ruta:** Ordenar lógicamente las coordenadas extraídas para trazar una ruta continua que el robot pueda seguir sin movimientos bruscos o innecesarios.
- **Definición de way-points:** Traducir este mapa de puntos en una secuencia estricta de way-points (puntos de paso) que garantizarán que el actuador o robot dibuje los rasgos del felino con la mayor fidelidad posible.

Fase 3: Implementación y Comparativa de Control

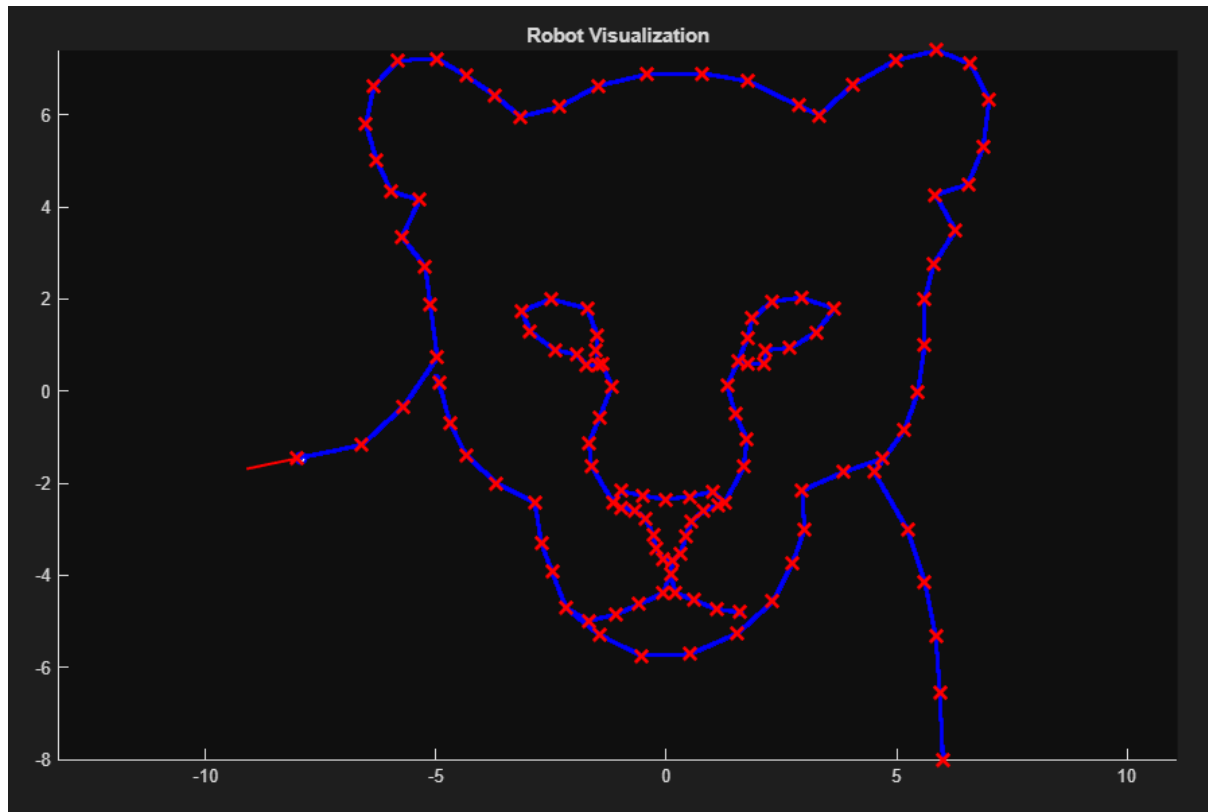
- **Desarrollo del algoritmo:** Programar el código de seguimiento de trayectoria en el entorno de desarrollo correspondiente.
- **Comparación de técnicas:** Implementar y contrastar el desempeño de dos métodos de seguimiento: la navegación orientada por way-points (para asegurar el alcance de cada coordenada exacta) versus un esquema de control de lazo abierto (para evaluar el error acumulado en el dibujo al no tener retroalimentación).
- **Sintonización de parámetros:** Ajustar los parámetros cinemáticos y de control del robot móvil para optimizar el trazo. Esto incluye calibrar la velocidad lineal (v), la velocidad angular (w), el periodo de muestreo (t_s), el tiempo total de simulación (t) y las ganancias del sistema.

Resultados

Lazo abierto



WayPoints



Conclusión

La implementación y comparación de ambas técnicas de control revela una diferencia fundamental entre la teoría cinemática ideal y el comportamiento dinámico real (o simulado con físicas) de un robot móvil al ejecutar trayectorias complejas.

- **Control en Lazo Abierto:** Esta técnica opera bajo la suposición de un entorno perfecto. Al dictar las velocidades lineal y angular sin ningún tipo de retroalimentación, el sistema confía ciegamente en que el robot ejecutará el comando con precisión absoluta en el tiempo establecido. Sin embargo, este enfoque es altamente vulnerable a la acumulación de errores de odometría. Factores como el deslizamiento de las ruedas, la fricción o pequeños desajustes de tiempo provocan que el error crezca exponencialmente. Aunque en la simulación, se vea un dibujo ideal, al trasladarlo a un escenario real, el error se acumularía iteración tras iteración generando una figura completamente diferente.
- **Navegación por Waypoints:** A diferencia del lazo abierto, este método iterativo (como el algoritmo Pure Pursuit) se basa en la corrección continua. El controlador de MATLAB evalúa constantemente la posición actual del robot frente a la coordenada de destino (way-point) y ajusta dinámicamente las velocidades cinemáticas permitidas para corregir el rumbo. Al considerar las limitaciones físicas y las velocidades reales del robot, este enfoque garantiza que la máquina alcance cada punto del mapa espacial, compensando cualquier perturbación o desviación en el camino.